RED EDEN · INSTALLATION GUIDE

Integrating RED EDEN with Unity Third Person

Inventory, interaction, crafting, resources, agricultural silo, and ARES Gemini Live

> **Goal:** Import RED EDEN systems into another Unity project and connect them around its existing player without replacing movement or camera systems.

Reference editor: `Unity 6000.3.16f1`

Package: `packages/RedEden-Full-System.unitypackage`

This guide does not claim Gemini Live connectivity; check the repository's release and test status before use.

1. Requirements and import

- Unity 6 compatible with the source project, and Unity Input System 1.19.0 or a compatible later version.
- A licensed Devion installation: Inventory System, Stat System, Triggers, Utilities, and UI Widgets. Install them from the licensed account. RED EDEN does not include Devion source, UI, or samples.
- Keep the target project's Unity Starter Assets — Third Person Controller. ARES is optional and needs the Gemini Live backend.

1. Back up the target project and install required licensed dependencies first.
2. Merge the Input System and URP dependencies from `ARES_INTEGRATION_PACKAGE/Unity/manifest-dependencies.json` into the existing `Packages/manifest.json`; do not replace the manifest.
3. In Unity, choose `Assets > Import Package > Custom Package...`, select `RedEden-Full-System.unitypackage`, and import its contents.
4. Wait for compilation and check the Console. Devion assembly errors usually mean that the licensed modules are missing or incompatible; do not import duplicate Devion files.

> The package does not import the test scene or a mandatory player and does not replace movement or camera systems. Do not combine a full-package import with manual folder copying in the same project.

Verify databases

Confirm that `RED EDEN Core Inventory Database.asset`, `RED EDEN Item Database.asset`, and `RED EDEN Crafting Database.asset` exist. Open them in the Project window and check that their data appears without missing references. Preserve all `.meta` files and do not move assets after import.

2. Existing player, inventory, and interaction

Preserve the current player

1. Keep the existing `CharacterController`, `ThirdPersonController`, `PlayerInput`, camera controller, and `AudioListener`.
2. Do not add another player or movement controller. Set the player Tag to `Player` when required by the Devion integration.
3. Keep one Devion `InventoryManager` in an existing or dedicated manager object; do not duplicate it.

Connect inventory

1. Assign `RED EDEN Core Inventory Database.asset` as the InventoryManager main database.
2. Add `RED EDEN Item Database.asset` as a child item database. Keep other game databases as needed and avoid duplicate RED EDEN entries.
3. Use the inventory UI from your licensed Devion installation. Add an `EventSystem` only if the scene does not already have one and the UI requires it.

Interaction

Use the existing station and node components with the Devion interaction layer; do not build a competing interaction framework. Existing RED EDEN components use `E` where configured. If interaction or prompts fail, inspect player Tag, Collider/Trigger setup, physics layers, and Active Input Handling.

> Do not assume every project has identical UI or interaction components. Connect items through the Inspector using fields actually present in your licensed Devion version.

3. Crafting, resources, and silo

Crafting

1. Place a crafting-station prefab or the existing `RedEdenCraftingStation` component.
2. Assign `RED EDEN Crafting Database.asset` and verify recipe/item references.
3. Test an available and unavailable recipe. Unavailable recipes should be visibly disabled, and ingredients should be deducted only when crafting starts.
4. Test a full inventory; finished output must remain available, and each station must retain its own state.

Resource nodes

Place the appropriate prefab from `ResourceNodes`, then verify its resource definition, remaining quantity, and model/material references. Collect a resource and reload the scene to check the project's persistence behavior.

Agricultural silo

1. Use the production prefab `Hydroponic Farm M01 - Agricultural Silo.prefab`; do not replace it with a temporary model.
2. Verify `Hydroponic_Farm_M01.fbx`, its materials, and crop/oxygen database references are present.
3. Inspect the `HydroponicFarm` component, production settings, inventory, and database references before placing it in the scene.
4. Run a complete production cycle, then collect the output and test a full inventory. Avoid editing the source prefab during testing.

> If the model or script is missing, verify that import completed and licensed Devion dependencies are installed before rebuilding the prefab or deleting files.

4. ARES Gemini Live setup (optional)

1. Add ARES components to the existing player or a separate object and assign their Inspector references. Do not add a new movement or camera system.
2. Set the Gemini Live configuration Backend URL to `https://red-eden.onrender.com`. Never place an API key or token in Unity or the project.
3. Assign `ARESMicrophoneVoiceInput` as the streaming microphone provider; input is mono 16 kHz PCM. Assign `ARESAudioPlayer` and an AudioSource; response output is 24 kHz PCM.
4. Set the language in `ARESSystemConfiguration` to `Arabic` or `English`, or provide it through `IARESGameSettingsProvider`. Hold **V** to speak and release it to end transmission, according to the current input binding.
5. Grant microphone permission when requested. Audio is sent to the backend and then Google Gemini Live; obtain any required user consent.

> A successful `/health` response does not prove that WebSocket or Gemini Live sessions work. The public service may need redeployment; do not claim live voice works until an authenticated WebSocket session is verified without exposing secrets.

Privacy and security

Keep the Gemini API key and session-signing key on the server only. Never log keys, session tokens, or raw audio. ARES should only read supported game-state summaries and must not be granted authority to change inventory or gameplay state.

5. Integration checks and troubleshooting

1. Run the scene and confirm movement and camera behavior are unchanged.
2. Verify there is one inventory manager and the correct main/child databases.
3. Test interaction, item pickup, available/unavailable recipes, a full inventory, and two independent stations.
4. Verify the silo prefab, FBX, and materials, then test production and collection.
5. Run available EditMode tests. Test ARES separately from core gameplay systems.

Problem	First check
Devion type or missing-assembly errors	Install compatible licensed Inventory, Stat, Triggers, Utilities, and UI Widgets modules.
Missing database or item	Check .meta/GUID files, main/child database assignments, and recipe references.
Interaction does not work	Check player Tag, Collider/Trigger, physics layers, and Input System.
Silo prefab is broken	Reimport the complete model/material assets; do not replace the prefab with a placeholder.
ARES cannot connect	Check backend URL, server configuration, session validity, and network access; never put secrets in Unity.
No audio	Check microphone permission, AudioSource, 16/24 kHz PCM rates, and platform WebSocket support.

This guide is text-only; it does not contain unverified screenshots. See the repository README, DEPENDENCIES, and INSTALLATION-CHECKLIST for current release status.